FINAL 90-MINUTE LECTURE

Deep neural networks, CNNs and RNNs

From vector inputs to images and sequences: how architecture matches data structure.

Anush Sankaran

00 / FLOW

The lecture follows the architecture decision path.

1
DNN
What a deep neural network is.

2
Example
A real-world DNN forward pass.

3
Encoder-decoder
A brief refresher.

4
CNN
Images need filters and feature maps.

5
RNN
Sequences need state and memory.

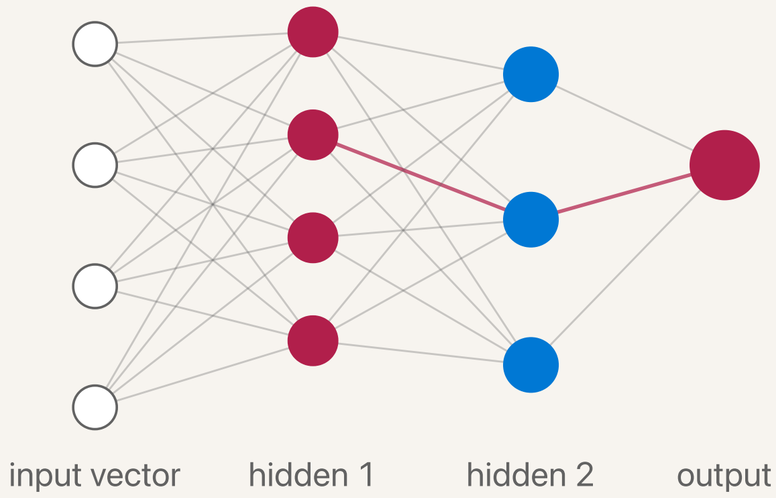
A deep neural network is a stack of learned transformations.

Each layer turns one vector into another vector.

$$h^{(1)} = \sigma(W^{(1)}x + b^{(1)})$$

$$h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)})$$

$$\hat{y} = \text{head}(h^{(L)})$$



01 / REAL-WORLD EXAMPLE

Example: a DNN scores whether a transaction is fraudulent.

INPUT

A transaction becomes a vector.

$248

11:42 PM

new city

merchant type

device age

past behavior

OUTPUT

0.91

Estimated fraud risk; the decision threshold is a product choice.

01 / VECTOR INPUT

The first design step is turning the world into numbers.

NUMERIC

amount

Scale or normalize.

CATEGORICAL

merchant

One-hot or embedding.

BEHAVIORAL

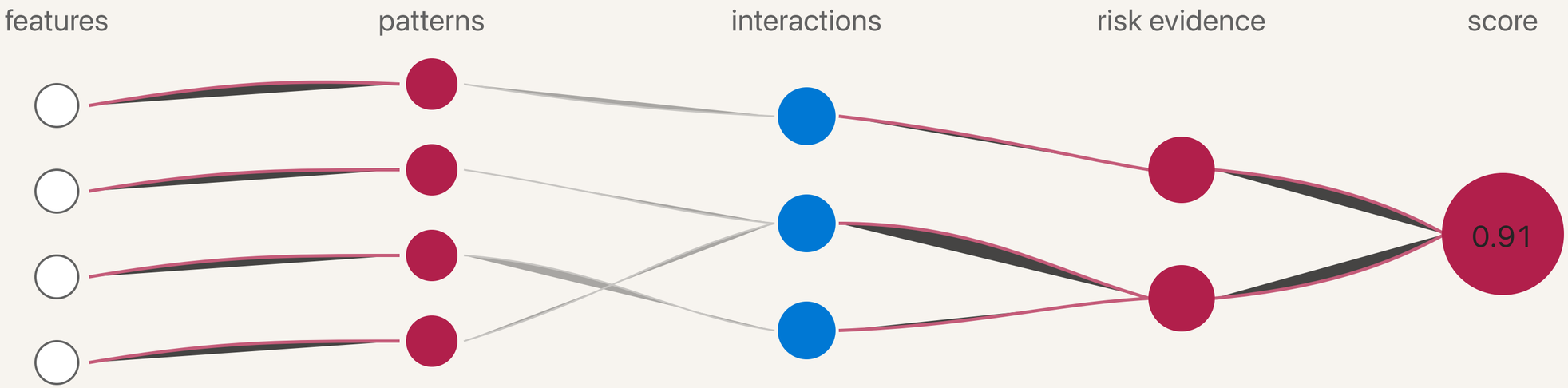
history

Aggregates over time.

A standard DNN expects a fixed-length vector x.

01 / FORWARD PASS

A forward pass is evidence flowing through layers.



01 / HIDDEN FEATURES

Hidden units become learned feature detectors.

Late night + new city

A useful interaction, not a raw input column.

Amount unusual for user

A learned comparison against behavior.

Merchant-risk pattern

A representation shaped by labels.

01 / TRAINING

Training adjusts every weight to reduce prediction error.

loss = L(ŷ, y)

Compare the predicted risk with the known outcome.

weights ← weights − η ∇loss

Backpropagation assigns credit across all layers.

01 / LIMITATION

But a dense DNN expects a vector.

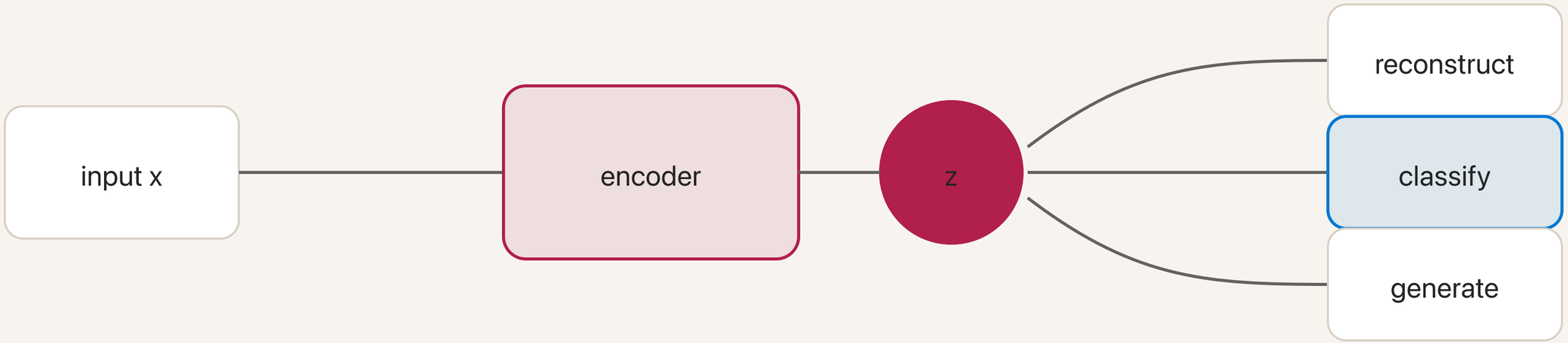
$x \in R^d$

One fixed-length list of numbers.

Images are not just lists. Language is not just a bag of words.

Encoder-decoder models separate remembering from producing.

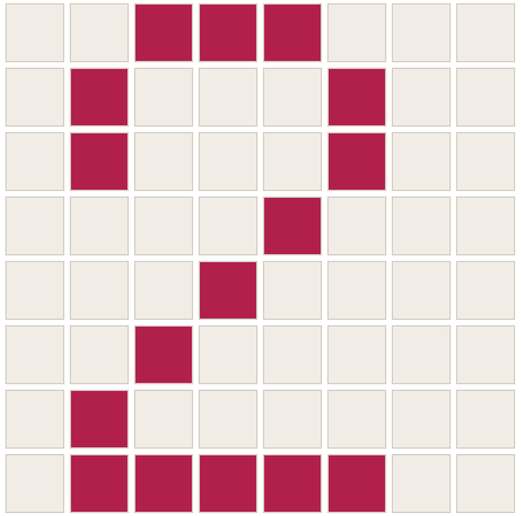
The same representation can feed different outputs.



If the input is an image, keep the grid.

03 / WHY NOT FLATTEN?

Flattening hides the neighborhood structure.



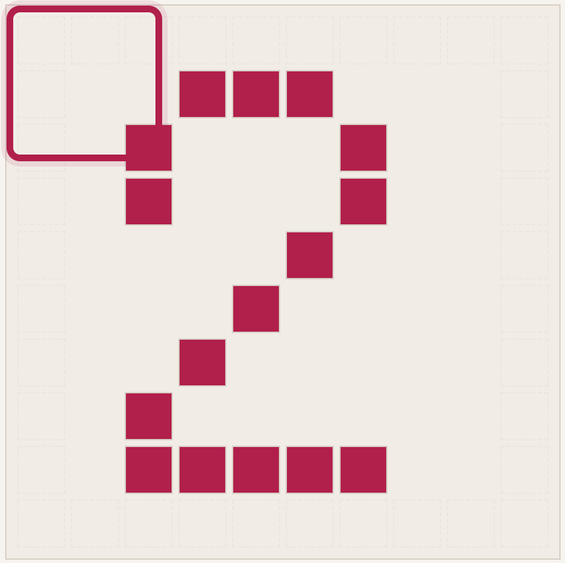
Pixels next to each other form strokes, edges, corners.

A CNN sees small neighborhoods first, then combines them into bigger patterns.

03 / FILTERS

Convolution is a sliding-window dot product.

ZERO-PADDED INPUT IMAGE



position 1 / 64

VERTICAL-EDGE FILTER

-1	0	1
-1	0	1
-1	0	1

Pause | Step back | Step next | Reset

Speed

sum = 0

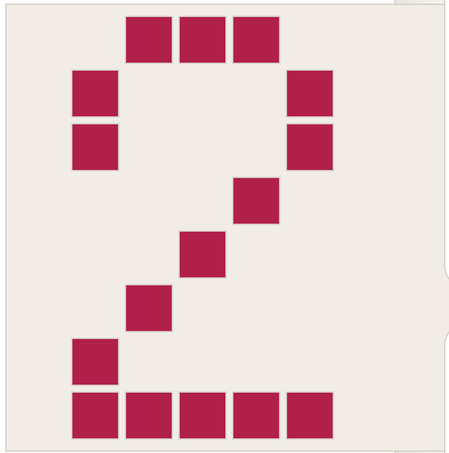
OUTPUT FEATURE MAP

Each number is one filter response.

03 / CLASSICAL FILTERS

The same image produces different feature maps.

SHARED INPUT

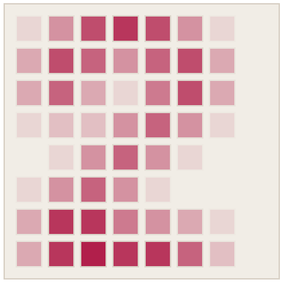


Every filter below is applied to this exact image.

GAUSSIAN BLUR

1	2	1
2	4	2
1	2	1

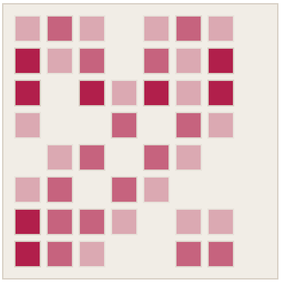
OUTPUT



SOBEL X

-1	0	1
-2	0	2
-1	0	1

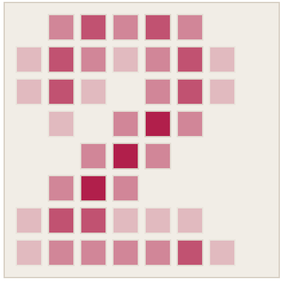
OUTPUT



LAPLACIAN

0	-1	0
-1	4	-1
0	-1	0

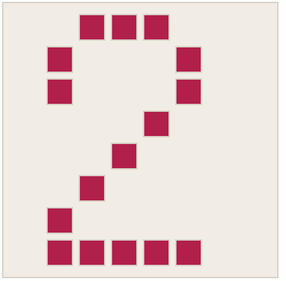
OUTPUT



SHARPEN

0	-1	0
-1	5	-1
0	-1	0

OUTPUT

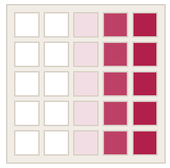


03 / LEARNED FILTERS

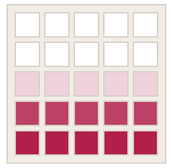
Handcrafted filters are named; learned filters are discovered.

Handcrafted filters

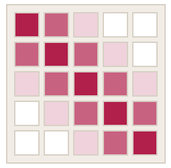
Designed by people to detect a known visual property.



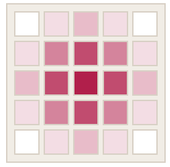
vertical edge



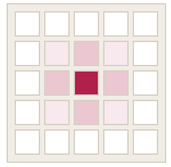
horizontal edge



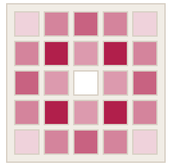
diagonal edge



blob



center spot

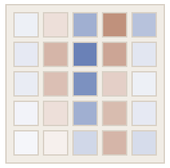


surround

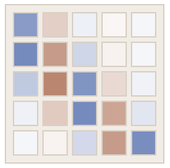
We choose the visual question before training.

Learned CNN filters

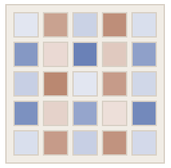
Weights start random, then training shapes them into useful detectors.



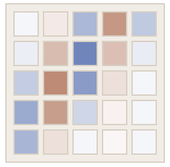
stroke fragment



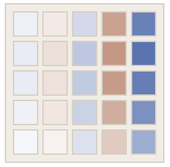
corner-like



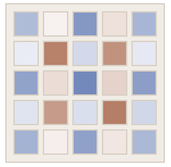
texture mix



curved trace



dark-to-light



task-specific

The model chooses the visual questions that reduce loss.

03 / CNN ARCHITECTURE

A CNN learns visual features in stages.

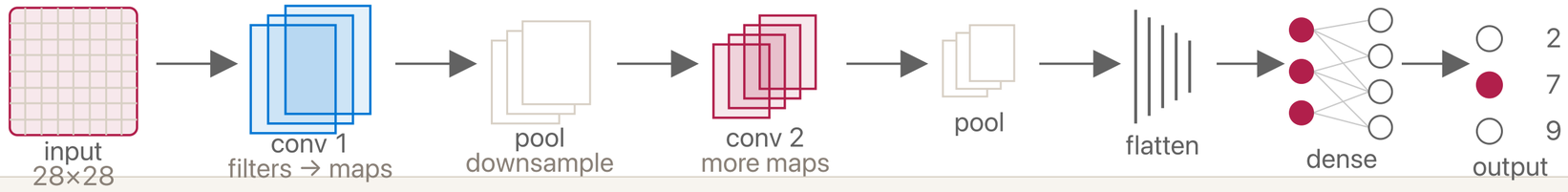


Image grid

Learned filters

edge | blur

corner | stroke

Feature maps

EDGES

INK

CHANGES

Pool + compose

Combine maps into larger visual parts.

Classifier

2 .18

5 .27

7 .82

Pause | Back | Next | Reset | SPEED

Start with the image as a grid of pixels.

03 / REAL CNN EXAMPLE

LeNet-5: a real CNN for handwritten digits.

Input
32×32 image

→

C1
5×5 conv
6 maps · 28×28

→

S2
2×2 avg pool
6 maps · 14×14

→

C3
5×5 conv
16 maps · 10×10

→

S4
2×2 avg pool
16 maps · 5×5

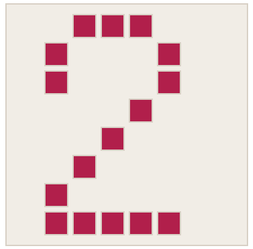
→

C5 / F6
120 → 84
units

→

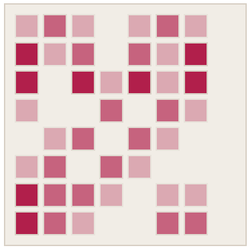
Output
10 digit
classes

Input digit



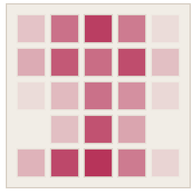
raw pixel grid

C1



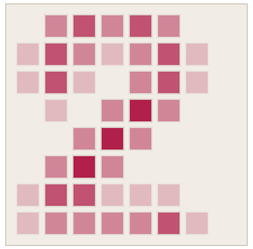
early edge responses

S2



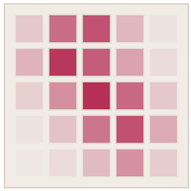
pooled, lower-resolution map

C3



stroke and change detectors

S4



compact spatial code

Classifier

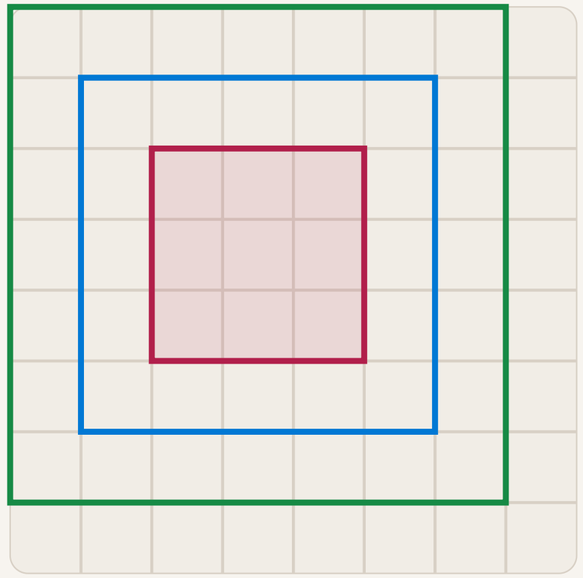


7

Source: LeCun, Bottou, Bengio & Haffner, “Gradient-Based Learning Applied to Document Recognition,” Proceedings of the IEEE, 1998.

03 / RECEPTIVE FIELD

Stacking filters lets later units see larger regions.



layer 1

layer 2

layer 3

Small filters; large understanding through depth.

Build a CNN by repeating visual feature extraction, then classify.

1. Convolution + activation

Learn many local filters and feature maps.

2. Downsample or stride

Reduce resolution and expand context.

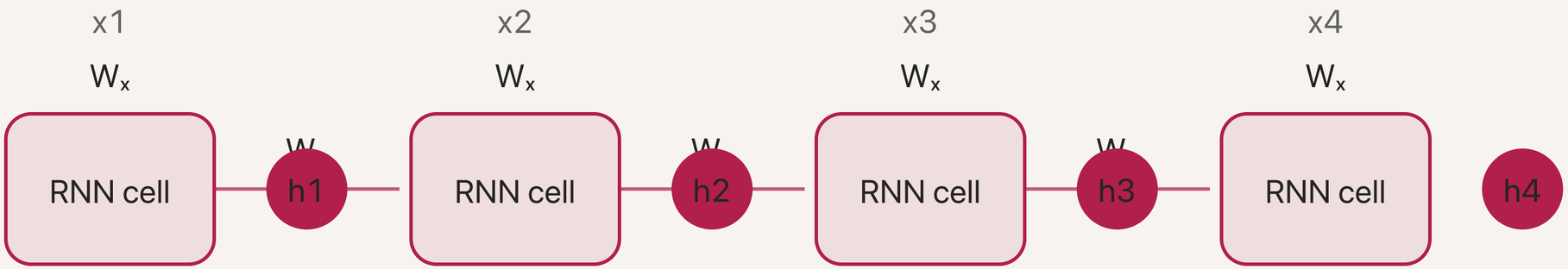
3. Pool + head

Turn final maps into a vector for prediction.

CNN = image-aware encoder + task head.

Language, speech and time series need memory.

An RNN reuses the same cell across a sequence.



same weights, updated hidden state

$$h_t = \sigma(W_x x_t + W_h h_{t-1} + b)$$

04 / SEQUENCE EXAMPLES

The same RNN idea applies to different sequence data.

LANGUAGE

words or tokens

Context changes meaning: "not good".

SPEECH

audio frames

Phonemes unfold over time.

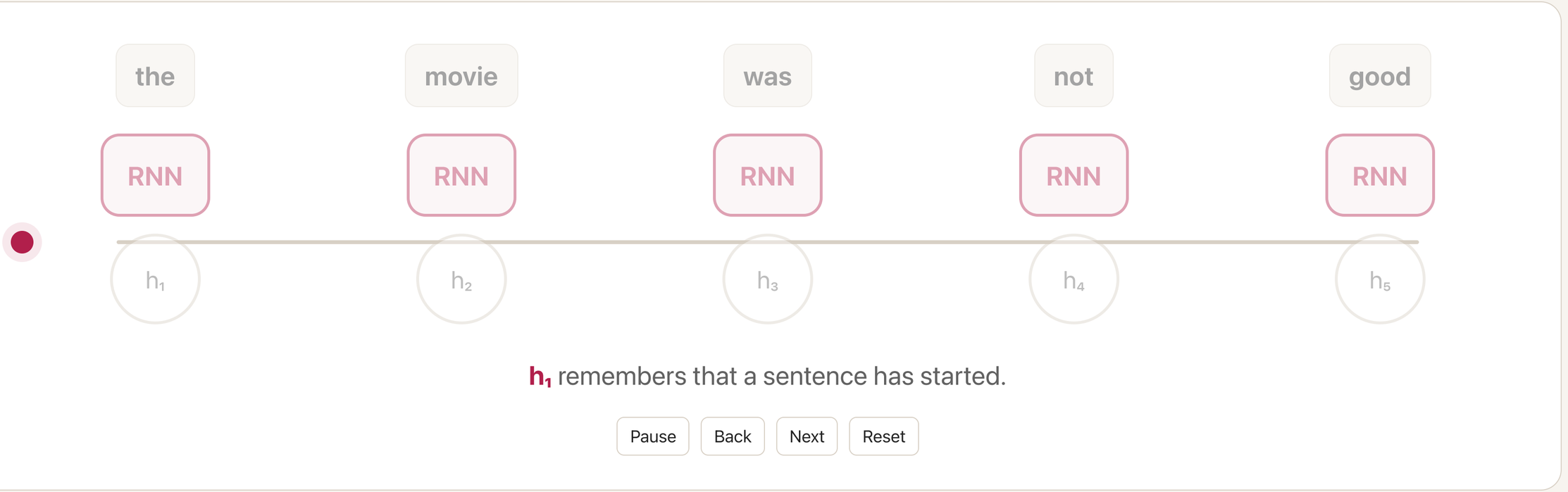
TIME SERIES

measurements

Past values shape future predictions.

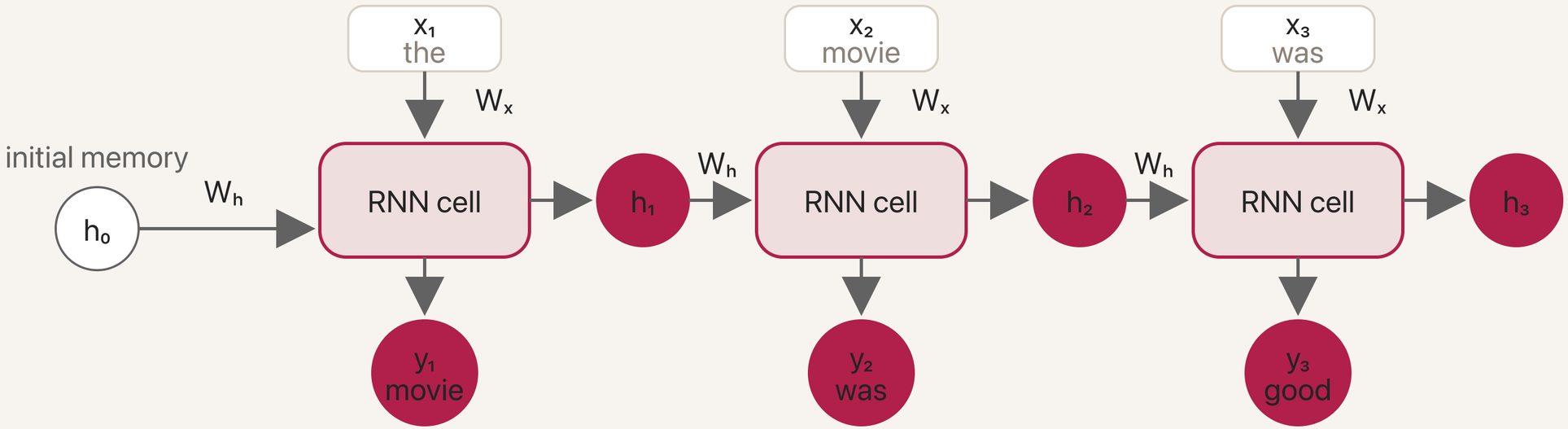
04 / LANGUAGE

For language, the hidden state carries previous words.



At each time step, an RNN consumes input, updates memory, and emits output.

$$h_t = f(W_x x_t + W_h h_{t-1} + b) \quad \text{and} \quad y_t = g(h_t)$$

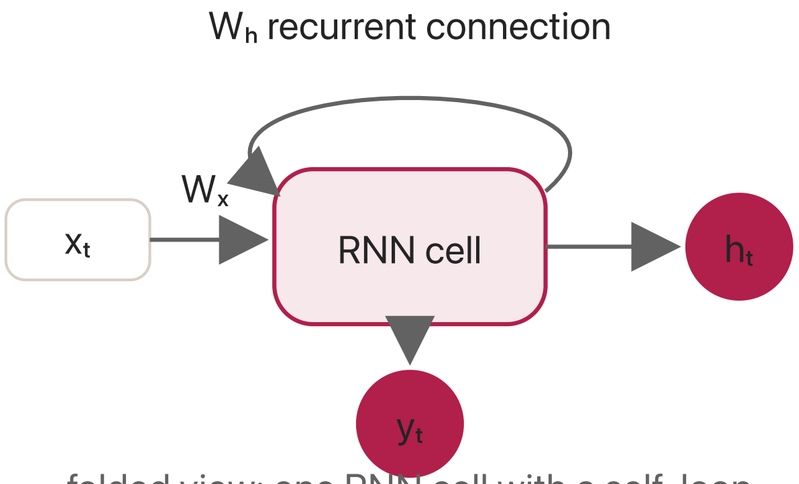


"the movie was good" → y_1 predicts "movie", y_2 predicts "was", y_3 predicts "good"

loss = Σ_t CE(y_t, true next word)

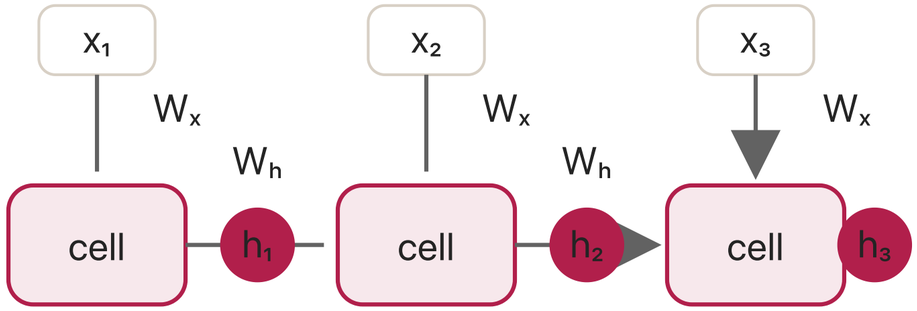
The same RNN can be drawn folded or unfolded through time.

Folded view



folded view: one RNN cell with a self-loop

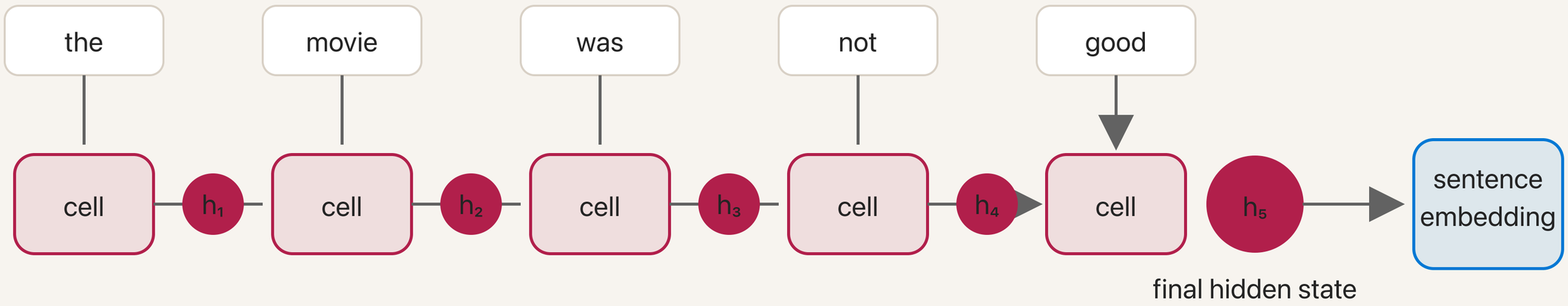
Unfolded view



unfolding copies the same cell and weights across time

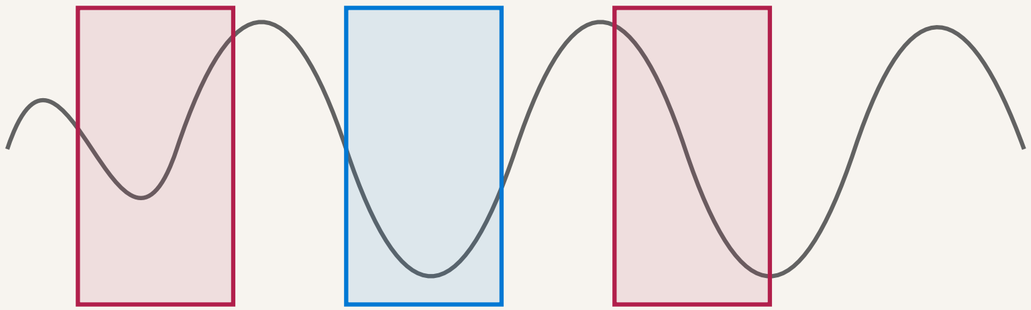
04 / SENTENCE EMBEDDING FROM FINAL STATE

The last hidden state can summarize the whole sentence.



For many sentence-level tasks, h_5 is fed to a classifier because it has accumulated information from all tokens.

For signals, each step is a frame or measurement.



audio frames -> x1, x2, x3...

RNN state lets the model smooth, forecast, or decode over time.

04 / GATED RNNS

LSTM and GRU add gates for longer memory.

Plain RNN

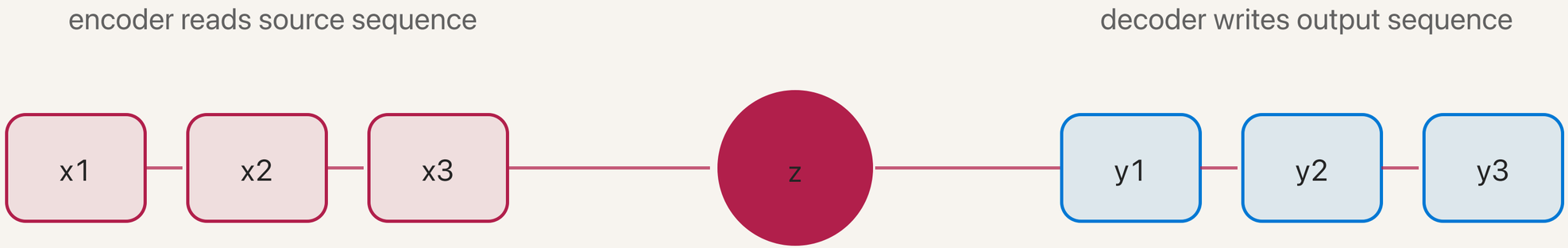
One hidden state update can forget too quickly.

Gated RNN

Learned gates decide what to keep, write, and expose.

04 / RNN ENCODER-DECODER

RNNs can encode one sequence and decode another.



Translation, captioning, speech-to-text and forecasting use this pattern.

05 / CHOOSING MODELS

Pick the architecture by the structure you need to preserve.

VECTOR

DNN

Fixed-length features.

GRID

CNN

Spatial neighborhoods.

SEQUENCE

RNN

Ordered memory.